TTIC 31180 — Final Project Proposal

Graphical Model Structure Learning for Sparse Feature Circuits

[Siddharth Raj] | April 23, 2026

1. Problem

Sparse autoencoders (SAEs) have become a central tool in mechanistic interpretability for decomposing transformer activations into interpretable features. Given such a decomposition, a natural follow-up question is how features in one layer causally influence features in later layers — the structure of what the community calls a *feature circuit*.

The current standard method, **attribution patching** (Marks et al., 2024), uses gradient-based approximations to score candidate edges. It scales well, but has known limitations: it produces edge scores rather than a graph structure, relies on local linearity, and does not cleanly separate direct from indirect effects. If $A \to B \to C$, attribution may score $A \to C$ highly even when no direct edge exists.

Distinguishing direct from indirect dependencies is exactly the problem classical **graphical model structure learning** was designed to solve. We would like to explore whether principled structure-learning methods — score-based search and constraint-based algorithms grounded in conditional independence — can recover feature circuits that are more causally faithful than what attribution patching produces.

2. Relation to course material

The project sits squarely in the structure-learning portion of the PGM curriculum. Feature circuits are DAGs over random variables (feature activations), so the setup ties directly to Lectures 2–3 on Bayesian networks and independence. We expect to use score-based methods with decomposable scores such as BIC (Koller & Friedman, Ch. 18) and the constraint-based PC algorithm, which applies d-separation reasoning to recover the Markov equivalence class. Since SAE features tend to be near-binary and sparse, we will model local conditionals as log-linear distributions, drawing on the Lecture 5 treatment of feature-based parametrizations. We may also include NOTEARS (Zheng et al., 2018) as a continuous-relaxation comparison, which connects to the Lecture 10 theme of casting discrete problems as continuous optimization.

3. Approach

We plan to study this on **induction circuits** (Olsson et al., 2022) as a starting point. These are among the simplest and best-characterized circuits in transformers, which makes them a good testbed: the head-level ground truth is known, and the feature-level circuit is expected to be small enough that classical structure-learning methods remain tractable. We intend to use a pretrained GPT-2 small together with publicly available pretrained SAEs, rather than training either from scratch.

The rough plan is to collect feature activations over a large number of induction-style prompts, filter to the features that appear task-relevant, and then apply several structure-learning methods to the resulting observational data. We would compare the recovered graphs against attribution patching and against each other, and — where the methods disagree — validate a subset of edges with direct activation patching as a causal ground truth.

Open questions we expect to work through as the project develops:

- How should the variable set be defined? Individual features, grouped features, or some kind of aggregated representation? This choice affects both the scientific story and the tractability of structure learning.
- What is the right conditional independence test for sparse, zero-inflated activation data? Standard tests may behave poorly here, and this may itself become a subproblem worth studying.
- Is induction a rich enough testbed, or do we need to move to something more structured like the IOI task (Wang et al., 2023) to see meaningful differences between methods?

4. Why this is a reasonable bet

Attribution patching is gradient-based and agnostic to conditional independence; the methods we propose are designed precisely to separate direct from indirect effects. If the graphical-model approaches outperform attribution patching against intervention-based ground truth, that is a result the interpretability community would care about. If they do not, characterizing *why* — whether because of data sparsity, identifiability limits, or something more fundamental — is itself informative. In either direction we expect to learn something.

5. Anticipated challenges

The project has several places where we expect to spend real effort. Sparse activation data is known to cause problems for standard CI tests, so we may need to adapt or replace them. Choosing the right granularity of features is nontrivial and may require iteration. Direct activation patching, which we want to use as ground truth, is expensive, so our causal validation will be a sample rather than exhaustive. Integrating pretrained SAEs, TransformerLens, and the attribution-patching reference code into a single pipeline will take some upfront engineering.

6. Evaluation

We expect the primary evaluation to be *intervention agreement*: for edges where methods disagree, how well does each method's prediction match the effect observed under direct activation patching? Secondary metrics would include recovery of the published head-level induction circuit, pairwise agreement between methods, and computational cost — since attribution patching's main appeal is speed, any principled alternative should be honest about the tradeoff.

7. Datasets and software

The main dataset is synthetic: a large set of induction-pattern prompts, trivially generated. If time permits, we would extend to the IOI dataset as a richer stretch task. On the software side, we expect to use PyTorch and TransformerLens for model access, pretrained SAEs from EleutherAI or OpenAI, the Marks et al. public code as an attribution-patching reference, and our own implementations of the structure-learning methods (implementing them is part of the learning value of the project). A single GPU should suffice for induction experiments.

8. Rough timeline

Week	Focus
1–2	Set up the pipeline, validate structure-learning code on synthetic DAGs, collect induction activations.
3–4	Run structure-learning methods, reproduce attribution-patching baseline, compare.
5	Intervention-based validation, ablations, begin stretch task if time allows.

If the SAE pipeline turns out to be more work than expected, we have a natural backup: run the same methodology at the attention-head level, which is a well-studied setting and does not depend on SAE availability.

9. What I hope to contribute

I am not aware of prior work systematically comparing classical graphical-model structure learning against attribution patching on feature circuits. Even a careful empirical comparison — with intervention-based validation — would be useful.